

Lab+ 总结报告

姓名：朱文凯

学号：23307110192

1. 总体目标

本次 Lab+ 的目标是在已经完成 Lab1 到 Lab6 主线 CPU 的基础上，继续补充可验证的 Bonus 功能，并尽量尝试运行更完整的 xv6。官方 Lab+ 说明中提到，Lab+ 可以补做之前的 Bonus，也可以完成官方新增方向，例如 xv6 主 Track、简单性能优化、原子指令和 PMP 等内容，参考：[Lab+ & Bonus](#)。

2. 已完成的主要工作

2.1 乘除法 Bonus

我补齐了 RV64M 整数乘除法，包括 `mul/mulh/mulhsu/mulhu`、`div/divu/rem/remu`，以及 `mulw/divw/divuw/remw/remuw` 等 W 版本指令。

实现上，我在 `common.sv` 中扩展 ALU 操作类型，在 `core_decode.sv` 中识别 `funct7 = 7'b0000001` 的 M 扩展指令，并在 `core_alu.sv` 中完成乘除法组合逻辑。边界语义包括除零、`INT_MIN / -1` 溢出和 W 指令符号扩展。

验证结果：

- `make sim` 构建通过。
- Lab1 extra 通过，关键输出为 `HIT GOOD TRAP at pc = 0x8002001c`。
- Lab4、Lab5、Lab6 回归保持通过。
- Lab+2 原先在 `mulw` 处的 `mismatch` 被清除。

2.2 Lab+2 microbench 正确性与性能统计

在实现乘除法后，Lab+2 microbench 已经能在 `DiffTest` 下持续运行，不再出现早期正确性 `mismatch`。默认随机延迟下，`qsort` 和 `queen` 通过；关闭仿真 RAM 随机等待后，`qsort`、`queen`、`bf` 通过，后续主要受性能限制。

为了分析性能瓶颈，我在 `core.sv` 中加入 Verilator 仿真期轻量性能计数器，并用 `BENCHMARK=1` 控制输出，默认构建不打印统计信息。统计项包括周期数、提交指令数、IPC、分支数、跳转数、控制流重定向、load-use stall、取指等待和访存等待。

50M cycle 基线采样中，IPC 约为 0.182，`fetch_waits` 约为 31.25M/50M，默认不跳转预测的分支方向正确率约为 19.5%。这些数据说明取指等待和控制流重定向是主要性能瓶颈。

2.3 简单分支预测

我实现了静态分支预测：

- backward branch 预测 taken。
- forward branch 预测 not taken。
- 条件分支在 ID 阶段计算预测目标。
- EX 阶段校验真实结果，预测错误时 flush 并 redirect。

当前没有实现 BHT、BTB、RAS，也没有提前处理 `jal/jalr`。

50M cycle 对比采样中，IPC 从约 0.181989 提升到约 0.190101，提升约 4.46%；分支预测正确率约 73.3%；EX 阶段控制流重定向从约 1.31M 降到约 0.73M。这说明简单静态预测对当前 `microbench` 有明确收益。

验证结果：

- `make sim` 构建通过。
- Lab+2 采样无 `Diffptest mismatch`。
- Lab4、Lab5、Lab6 回归保持通过。

2.4 原子指令

我先实现了 Lab+3 `atomicity.bin` 所需的最小集合：

- `amoswap.w`
- `amoadd.w`
- `lr.w`
- `sc.w`

随后继续补全官方列出的 32-bit AMO RMW 指令：

- `amoxor.w`
- `amoand.w`
- `amoor.w`

- `amomin.w` / `amomax.w`
- `amominu.w` / `amomaxu.w`

实现上，我在译码阶段识别 AMO opcode，在流水线 packet 中传递 AMO 元信息，并在 MEM 阶段用两步 RMW 完成 AMO：先读取旧 word，再计算新 word 并写回。写回 rd 的值按 RV64 规则对旧 word 符号扩展。lr.w/sc.w 使用单核 reservation 地址记录；sc.w 成功时写内存并返回 0，失败时不写内存并返回 1。

验证结果：

- `make test-labplus-3` 通过，关键输出为 `HIT GOOD TRAP at pc = 0x800000dc`。
- 新增完整 AMO W 自测通过，关键输出为 `HIT GOOD TRAP at pc = 0x800001c4`。
- Lab4、Lab5、Lab6 回归保持通过。

当前边界：未实现 64-bit AMO D 指令，未接入 `DiffTestAtomicEvent`，也未处理多核一致性问题。本项目为单核 CPU，本轮实现已覆盖当前 Lab+3 与报告所需的 32-bit AMO 范围。

2.5 PMP 权限检查

原先代码只支持 `pmpaddr0/pmpcfg0` 的 CSR 基本读写。我本轮进一步实现了 PMP entry0 的 NAPOT 权限检查，使 PMP 真正参与取指和访存权限判断。

实现范围：

- 支持 `pmpaddr0` 和 `pmpcfg0[7:0]` 的 entry0。
- 支持 NAPOT 区域匹配。
- 支持 R/W/X 权限检查。
- U/S-mode 受 PMP 限制，M-mode 默认不受限制。
- 暂不实现 TOR、NA4、多个 PMP entry 和 L-bit 锁定语义。

IF 阶段对取指地址检查 X 权限，EX 阶段对 load/store/AMO 地址检查 R/W 权限。违规时产生 `instruction/load/store access fault`，并复用现有 WB 阶段精确异常提交路径。

验证结果：

- `make sim` 构建通过。
- Lab+4 前置特权/PMP 子测通过，输出 `Single test passed.`。
- Lab+4 后续 benchmark 能继续运行到更后阶段，未再出现早期 PMP/access fault 阻塞。
- Lab+3、Lab4、Lab5、Lab6 回归保持通过。

2.6 MMU 缺页异常

Lab6 报告中曾说明未实现缺页异常。本次 Lab+ 中我补充了 MMU page fault 核心路径。

实现内容：

- 增加 instruction/load/store page fault cause。
- IBus/DBus/CBus 请求携带访问类型。
- MMU 响应增加 page_fault 标志。
- Sv39 页表遍历中检查 invalid PTE、W && !R、R/W/X/A/D 权限、基本 U-mode 权限和 superpage PPN 对齐。
- page fault 后不发起最终物理访存。
- core 在 IF/MEM 侧把 MMU fault 转成对应异常，并设置 faulting virtual address 到 mtval。

验证结果：

- make sim 构建通过。
- Lab4、Lab5、Lab6、Lab+3、Lab+4 关键回归保持通过。
- 新增 page fault 诊断生成器。load page fault 诊断在 Diffptest 下可见 DUT 产生 mcause = 13、mepc = 0x80000084、mtval = 0x40000000。

当前边界：page fault 核心硬件路径已完成，但 no-diff good-trap 自测收尾仍不稳定；MXR/SUM 语义没有完整实现。

2.7 S-mode trap、delegation 和 sret

原先 S-mode CSR 已有基本读写。本次 Lab+ 中，我进一步实现了 S-mode trap 生态闭环：

- 支持 sret 译码和提交。
- 支持 medeleg/mideleg 控制异常或中断进入 M-mode 或 S-mode。
- S-mode trap 写入 sepc/scause/stval，并跳转 stvec。
- sret 根据 sstatus.SPP/SPIE/SIE 恢复特权级和中断使能状态。

实现上，trap 目标选择集中在 WB 提交边界，由 core_trap_ctrl.sv 根据当前特权级和 delegation CSR 决定进入 M-mode 还是 S-mode。CSR 模块对应更新 M-mode 或 S-mode 的 trap 状态。

验证结果：

- make sim 构建通过。
- Lab5 extra S-mode bonus 通过，关键输出为 HIT GOOD TRAP at pc = 0x800002b4。

- 该测试覆盖 `mret -> S-mode -> sret -> U-mode ecall -> delegated S-trap -> sret`。
- Lab4、Lab5、Lab6、Lab+3、Lab+4 回归保持通过。

当前边界：暂未实现 `TSR/TW/TVM` 的非法化语义，`MXR/SUM` 也没有完全接入 `MMU` 权限判断；`S-level timer interrupt` 的 `pending/priority` 仍需后续细化。

2.8 xv6 主 Track 尝试

完整 `xv6` 是本轮最高成本任务。说明中也提到，该任务很难，即使只实现 `difftest` 中读取虚拟硬盘镜像也可以算作部分进展。

我先确认了当前仓库现状：

- `make test-lab5` 运行的是课程裁剪版 `xv6 kernel`。
- 该镜像能输出 `xv6 kernel is booting` 和 `Return from init! Test passed`。
- 仓库原本没有完整 `xv6` 源码、完整 `kernel.bin` 或 `fs.img`。
- 仿真设备原本没有可供 `xv6` 使用的块设备 `MMIO`。

本轮完成的 `xv6` 相关工作：

- 在 `emu` 中增加 `--fs-image` 参数。
- 实现 `load_ram_image_at()`，把第二镜像加载到固定物理地址 `0x87000000`。
- 新增 `RAM disk magic` 裸机测试，验证 `CPU` 能从 `0x87000000` 读取 `host` 加载的镜像内容。
- 引入可修改的 `upstream xv6-riscv` 源码到 `third_party/xv6-riscv/`。
- 增加 `make labplus-xv6-build`，生成 `xv6-kernel.bin` 和 `xv6-fs.img`。
- 安装 `RISC-V GCC/binutils` 后，成功构建 `xv6`。
- 将 `xv6` 的 `virtio disk` 路径替换为 `RAM disk` 内存拷贝。
- 适配 `UART` 地址，裁剪 `PLIC` 和 `SSTC timer` 路径。
- 将 `xv6` 编译目标从 `rv64gc` 改为 `rv64g`，避免当前 `CPU` 不支持的 `RVC` 压缩指令。
- 恢复 `mret -> S-mode main` 路径，避免 `M-mode` 忽略 `satp` 后访问 `xv6` 高地址 `kernel stack`。

验证结果：

- `RAM disk magic` 测试通过，关键输出为 `Loaded 4 bytes ... to 0x87000000` 和 `HIT GOOD TRAP at pc = 0x80000028`。
- `make labplus-xv6-build` 成功导出 `xv6-kernel.bin` 和 `xv6-fs.img`。
- 完整 `xv6` 适配镜像已经能输出 `xv6 kernel is booting`。
- 恢复 `S-mode mret` 后，运行可推进到用户虚拟地址 `pc = 0x0` 附近。

当前边界：仍未看到 `init: starting sh`，也没有进入可交互 `shell`。下一层阻塞应在用户态执行、`ecall / trap` 返回、`console` 文件或 `RAM disk` 文件系统路径中继续细分。本轮 Lab+ 到此终止，不继续深挖。

2.9 I-cache 性能优化

在性能优化方向上，我最终实现了一个最小 I-cache。

设计选择：

- I-cache 插入在 MMU 之后，使用物理地址。
- 只缓存取指请求。
- `load/store`、`AMO`、页表遍历和 `MMIO` 全部 `bypass`。
- `satp` 或特权级变化时保守 `flush`。
- 暂不实现 `D-cache`、`prefetch` 和完整 `fence.i`。

这样做的原因是：性能统计显示 `fetch_waits` 是主要瓶颈，而 `D-cache` 会引入 `store`、`MMIO`、`AMO`、`LR/SC` 和一致性风险。本轮优先选择低风险、可验证的 I-cache。

验证结果：

- `ReadLints` 无新增诊断。
- `make sim` 构建通过。
- `Lab4`、`Lab5`、`Lab5 extra`、`Lab6`、`Lab+3`、`Lab+4` 关键回归保持通过。
- `Lab+2 50M-cycle` 样本中，IPC 从此前约 `0.190` 提升到约 `0.307`，`fetch_waits` 从约 `31.2M/50M` 降到约 `18.5M/50M`。

当前边界：`cache line` 只有一个 `64-bit word`，未实现预取、`D-cache` 和完整 `fence.i`。

3. 原有 Bonus 的补充情况

3.1 已在原 Lab 或 Lab+ 中覆盖的项目

乘除法原本属于“任意 Lab 完成”的 Bonus，本次在 Lab+ 中补齐并通过 `lab1-extra`。

`Lab4` 中 `CSR` 作用说明、流水线刷新原因、`S-mode CSR` 基本读写，在原 `Lab4` 报告中已有说明。本次 Lab+ 继续把 `S-mode CSR` 从基本读写推进到 `trap/delegation/sret` 闭环。

`Lab5` 的巨页支持在原 MMU 实现中已有基础：页表遍历遇到上级 `leaf PTE` 时，会按当前 `level` 拼接物理地址，支持 `1GB / 2MB / 4KB` 映射。本次 Lab+ 没有重写该部分，但在总报告中保留说明。

Lab5/Lab6 中原先未完成的 MMU 缺页异常，本次 Lab+ 已补充核心硬件路径。

PMP 原先只有 CSR 基本读写，本次 Lab+ 补充了 `pmpaddr0/pmpcfg0 entry0` NAPOT R/W/X 权限检查。

Lab+ 官方新增的原子指令方向，本次已实现 `lr.w/sc.w` 和完整 32-bit AMO W 指令。

Lab+ 官方新增的简单性能优化方向，本次已实现性能统计、静态分支预测和最小 I-cache。

Lab+ 官方 xv6 主 Track，本次完成了启动尝试、RAM disk 加载链路、xv6 源码引入和部分 platform 适配，但没有进入 shell。

3.2 未选择或未完整完成的项目

Lab3 “使用非 Basys3 板子”没有选择，也没有上板验证材料，因此不作为本次 Lab+ 成果声明。

Lab6 “编写中断处理程序，在时钟中断时打印一些内容”没有作为单独 Bonus 完成。本项目 Lab6 主线已经支持机器时钟中断，并能通过 Lab6 测试，但本次没有额外编写一个独立的“时钟中断打印程序”作为 Bonus 材料。

Lab6 “思考时钟中断为什么使用 MMIO 计时器”原报告中没有专门回答。这里补充我的理解：MMIO 计时器把计时逻辑建模成外设，CPU 通过普通 load/store 访问 `mtime/mtimecmp` 等寄存器。这样核心流水线不需要内置复杂计时逻辑，仿真平台、SoC 和 FPGA 外设也能用统一方式连接计时器。计时器到期后通过中断线通知 CPU，CPU 再在 trap handler 中处理并设置下一次中断。

完整 xv6 shell、virtio-mmio、完整 PLIC、完整 S-level timer interrupt、D-cache、64-bit AMO D、多 PMP entry、TOR/NA4/L-bit、MXR/SUM 完整语义和上板扩展均未完成。本次报告只声明已经实现并验证到的边界。

4. 回归与验证总览

本轮 Lab+ 期间反复使用以下测试进行回归：

- `make sim`：构建通过。
- Lab1 extra：通过，HIT GOOD TRAP at pc = 0x8002001c。
- Lab4：通过，HIT GOOD TRAP at pc = 0x8001fff8。
- Lab5：通过，输出 Return from init! Test passed。
- Lab5 extra：通过，HIT GOOD TRAP at pc = 0x800002b4。
- Lab6：通过，输出 Privileged test finished. / Exit with code = 0。
- Lab+3：通过，HIT GOOD TRAP at pc = 0x800000dc。
- 完整 AMO W 自测：通过，HIT GOOD TRAP at pc = 0x800001c4。

- Lab+4: 前置特权/PMP 子测通过, 输出 `Single test passed.`。
- RAM disk magic 自测: 通过, `HIT GOOD TRAP at pc = 0x80000028`。

Lab+2 和 Lab+4 的完整长 benchmark 仍受性能和 cycle cap 影响, 没有把“完整跑完所有负载”作为本轮声明。xv6 主 Track 也只声明到 kernel banner 和用户入口附近, 不声明进入 shell。

5. 总结

本次 Lab+ 的主要成果是: 在五级流水线 CPU 上补齐了多个可验证的体系结构扩展, 包括 RV64M、32-bit AMO/LR/SC、PMP 权限检查、MMU page fault、S-mode trap/delegation/ sret, 并完成了性能统计、静态分支预测和最小 I-cache。

在系统方向上, 我尝试了完整 xv6 主 Track。最终没有进入 shell, 但完成了 RAM disk 加载链路、xv6 源码构建、RAM disk 驱动替换和平台适配, 并定位出当前边界: 必须使用 `rv64g`, 不能生成 RVC 指令; 必须走 S-mode `mret` 路径, 不能长期在 M-mode 直调 `main()`; 后续阻塞集中在用户态、系统调用、console 文件和 RAM disk 文件系统路径。